



Grenoble INP – ENSIMAG
École Nationale Supérieure d'Informatique et de Mathématiques Appliquées

Rapport de stage assistant ingénieur

Effectué au Laboratoire d'Ecologie Alpine (LECA)

Géoréférencement et segmentation de la neige de photos obliques de montagnes

Placice Neuilly
2e année – Option MMIS

21 mai 2026 - 31 juillet 2026

CNRS - Délégation Alpes
25 rue des Martyrs

BP 166
38042 Grenoble Cedex 9

Responsables de stage
Philippe Choler, Fatima Karbou, Guillaume James
Tuteur de l'école
Franck Rousseau

Sommaire

1 Introduction **3**
- 1.1 Contexte du stage . . . 3
- 1.2 Objectif du projet . . . 3
- 1.3 Données et environnement de test . . . 3
- 1.4 Etat de l'art . . . 4

2 Recalage des photos **5**

3 Traitement des images panoramiques **7**

4 Géoréférencement des photos **8**
- 4.1 Photogeoref par Meteoexploration . . . 8
- 4.2 Prise en main du logiciel et validation . . . 8

5 Segmentation de la neige **9**
- 5.1 Méthode classique par seuillage . . . 9
  - 5.1.1 Explication de l'algorithme . . . 9
  - 5.1.2 Analyse des résultats . . . 10
- 5.2 Deep Learning pour la segmentation sémantique . . . 10
  - 5.2.1 Annotation et préparation des données . . . 10
  - 5.2.2 Environnement de travail . . . 12
  - 5.2.3 Processus d'entraînement et d'évaluation . . . 12
  - 5.2.4 Prédiction . . . 13
  - 5.2.5 Améliorations possibles . . . 13

6 Rendu final **13**
- 6.1 Fonctionnement du script . . . 14
- 6.2 Paramétrage et exécutions isolées . . . 14

7 Analyse des résultats **15**
- 7.1 Évaluation au fil des saisons . . . 15
- 7.2 Analyse de l'algorithme de recalage . . . 16

8 Bilan personnel **17**

9 Conclusion et perspectives **18**

A Algorithme SIFT **20**

B Détails sur le processus d'entraînement du modèle de Deep Learning **20**
- B.1 Intersection over Union - IoU [8, 22] . . . 20
- B.2 Présentation de HRNet [23] . . . 21

1 Introduction

1.1 Contexte du stage

Ce projet a été encadré conjointement par trois laboratoires : le CEN (Centre d'Etudes de la Neige), le LJK (Laboratoire Jean Kuntzmann) et le LECA (Laboratoire d'Ecologie Alpine). Cependant, la quasi-totalité du travail a été effectuée au sein du LECA, étant le principal bénéficiaire par la finalité du stage. En effet, ce laboratoire étudie la biodiversité en montagne et le fonctionnement des écosystèmes alpins. Ainsi, comprendre l'évolution de la couverture neigeuse est essentiel pour étudier son impact sur la faune et la flore. Le CEN a apporté une expertise sur la nivologie et m'a permis de découvrir et d'étudier les méthodes déjà existantes qui pourraient me servir pour mon projet, tout en m'apportant des pistes à explorer lorsque j'étais bloqué. Finalement, le LJK fournissait la partie mathématique et informatique, en pouvant valider les techniques que j'ai utilisées.

1.2 Objectif du projet

Le stage a pour objectif, à partir de photos obliques de montagnes, de constituer un jeu de données comblant les images satellitaires qui ne passent pas tous les jours ou qui sont inutilisables à cause des nuages. Ce jeu de données devra être superposable avec ces images satellitaires et permettra d'avoir un suivi de l'enneigement plus précis. Obtenir ce jeu de données va permettre aux différents chercheurs de pouvoir analyser l'influence de la neige, notamment sur la biodiversité. Il faudra ainsi géoréférencer les photos (projeter les pixels sur les coordonnées réelles) puis classifier les pixels (blanc pour la neige, noir pour le reste). La méthode implémentée devra être transférable à plusieurs endroits.

La réalisation du projet s'est déroulée comme suit :

- Recalage des images pour pouvoir les superposer.
- Découpage des panoramas pour enlever les déformations.
- Création des masques neige/non-neige sur les photos découpées. (segmentation sémantique)
- Géoréférencement des masques.

1.3 Données et environnement de test

Pour réaliser cela, le projet va se décomposer en plusieurs parties, les algorithmes implémentés et/ou existants seront tous d'abord testés sur la webcam du site eLTER Lautaret Figure 1b (photo prise le 22 juillet durant une mission de terrain), prenant le panorama Figure 1a plusieurs fois par jour à intervalle régulier. Pour avoir un nombre raisonnable de photos, seulement un échantillon est étudié : du 14 juin 2023 au 23 octobre 2023, ce qui représente 125 images. Cette période a été choisie pour ses différents taux d'enneigement.

Cette cadence de prise nous permettra d'avoir quasiment tous les jours au moins une photo qui sera utilisable, sans nuage ou neige recouvrant toute la photo/caméra.



(a) Panorama de test, Lautaret



(b) Caméra de l'étude

Figure 1: Environnement de test

Ensuite des tests sur d'autres données seront effectués et l'automatisation des différentes étapes pour une prise en main plus directe sera réalisée.

1.4 Etat de l'art

L'utilisation de photographies terrestres obliques pour le suivi de la couverture neigeuse est un complément indispensable à l'imagerie satellitaire. Bien que les satellites offrent une couverture globale, l'exploitation de leurs images peut-être limitée par la couverture nuageuse. Les webcams terrestres comblent ce manque, mais introduisent des défis géométriques et photométriques pour pouvoir les exploiter et les comparer avec les images satellitaires. Il existe déjà de nombreux articles portant sur ces problèmes, que ce soit pour le géoréférencement, le recalage ou la segmentation sémantique des patchs de neige.

Géoréférencement

La transformation d'une image oblique 2D en une carte superposable à un Modèle Numérique de Terrain (MNT) est un problème fondamental en vision par ordinateur [21]. L'article fondateur de Corripio (2002) [2] a posé les bases du géoréférencement de photographies obliques de montagnes. Sa méthode consiste à projeter les points 3D d'un MNT sur le plan 2D de la caméra grâce à des matrices de transformation géométriques/trigonométriques. Pour résumer un peu sa méthode, les points du MNT sont mis dans un système de coordonnées centré sur la caméra, puis sont projetés sur un plan 2D représentant le capteur de l'objectif. Ensuite des décalages induits par la courbure de la Terre et la réfraction de la lumière sont corrigés. Pour assurer une précision optimale, des points de contrôle permettant d'associer certains points reconnaissables entre le MNT et la photographie peuvent être ajoutés.

L'inconvénient de cette méthode est qu'on suppose connaître tous les paramètres de la caméra (coordonnées précises, focale de l'objectif). Des articles comme [16] présentent une technique pour automatiser le processus en alignant les crêtes des montagnes, tout en évitant

de lacer des points de contrôle sur chaque photo. L'article de Corripio représente encore aujourd'hui la référence pour ce problème, la méthode décrite dans celui-ci est implémentée dans des outils comme Photogeoref [11], PRACTISE [6] ou bien Pic2Map [12].

Traitement géométrique

Le géoréférencement manuel de chaque image est chronophage, il faut l'automatiser. Cependant, les séquences temporelles de webcams subissent de micro-déplacements (vent, changements de température..) nécessitant un recalage préalable des images pour ensuite pouvoir appliquer le même géoréférencement pour chacune d'entre elles. Pour ce faire, les approches modernes s'appuient sur l'extraction de points d'intérêt robustes aux changements d'échelle et de lumière, méthode décrite par Lowe sous le nom d'algorithme SIFT (Scale-Invariant Feature Transform) [10, 14]. Afin de recaler des photographies en milieu montagneux, l'article [16] propose une approche basée sur l'algorithme SIFT. Une fois les points caractéristiques extraits, leur mise en correspondance entre deux images permet de déduire une matrice homographique, qui est ensuite appliquée sur l'image que l'on veut recaler.

Un autre problème est la déformation géométrique induite par les images panoramiques. Il est nécessaire de corriger les projections géométriques (par exemple, cylindriques) pour revenir à un espace rectilinéaire sans distorsion [5], sinon le géoréférencement sera faux car il suppose que les droites réelles restes droites sur l'image. Des logiciels comme FFmpeg [4] permettent de corriger cette déformation.

Segmentation de la neige

Le défi majeur du projet est la création des masques en noir et blanc classifiant la neige. Historiquement, les approches par seuillage sont les plus utilisées [2, 3, 6]. Ces méthodes reposent souvent sur la conversion de l'image vers d'autres espaces colorimétriques, comme le HSV ou le CIE LAB, couplés à des algorithmes d'égalisation d'histogramme (CLAHE) [19] pour être robustes face aux trop forts contrastes. Cependant, ces méthodes classiques sont limitées car sont non automatisables à plusieurs caméras ou même plusieurs photos à cause des seuils fixés. Ainsi, les dernières méthodes de classification de la neige s'orientent vers l'apprentissage profond (Deep Learning) pour la segmentation sémantique. Des architectures spécialement conçues pour ce problème, comme IceSnowNet [9] ou PyLC [13], prouvent que le Deep Learning permet de détecter à la fois les vastes étendues neigeuses et les petits patchs de neige, tout en évitant de les confondre avec d'autres éléments tels que les nuages. Cependant, la qualité des semgentations dépend fortement de la qualité de l'apprentissage réalisé. Ces modèles, incluant des réseaux à haute résolution comme HRNet [23] ou des Transformers comme SegFormer [7], nécessitent des données d'entraînement variées, telles que celles fournies par le MountainScape Dataset [17].

2 Recalage des photos

Comme expliqué précédemment, avant de géoréférencer les photos obliques et de les traiter en lot, il faut les recaler pour ne pas à faire pour chacune d'entre elles les calculs pour le géoréférence-

ment. En faisant cela, on va bien régler une photo, dite de référence puis on va appliquer le même géoréférencement pour toutes les autres [1] [21].

Fonctionnement de l'algorithme

L'algorithme de recalage implémenté est fortement inspiré de la méthode décrite par [16]. Son objectif principal est d'aligner un ensemble d'images obliques de montagne par rapport à une image de référence. Cette image de référence est sélectionnée au préalable pour ses caractéristiques optimales : une bonne luminosité et peu de nuages, permettant le bon fonctionnement de l'algorithme.

Le processus se déroule en deux grandes phases : l'initialisation de la référence et le traitement itératif des images cibles.

D'abord, l'image de référence est chargée en niveaux de gris. L'algorithme SIFT (*Scale-Invariant Feature Transform*) est instancié afin de détecter les points clés (ou *keypoints*) et de calculer leurs descripteurs associés (voir Annexe A pour plus de détails sur SIFT). Ces descripteurs serviront de base pour la mise en correspondance.

L'algorithme parcourt ensuite l'ensemble des photos du dossier d'entrée. Pour chaque image, les étapes suivantes sont exécutées de manière séquentielle :

- L'image courante est convertie en niveaux de gris, puis ses points clés ainsi que leurs descripteurs SIFT sont extraits.

- Les descripteurs de l'image courante sont comparés à ceux de l'image de référence. Cette recherche est optimisée grâce à un algorithme de correspondance basé sur des arbres k-d approximatifs via la bibliothèque FLANN (*Fast Library for Approximate Nearest Neighbors*), en configurant la recherche pour trouver les deux plus proches voisins ($k = 2$).

- Afin d'éliminer les fausses correspondances dues à des textures répétitives ou à des éléments mobiles (nuages, ombres), le test de ratio de Lowe est appliqué. La fonction de correspondance précédente renvoie, pour chaque keypoint de l'image en cours, les deux descripteurs de l'image de référence les plus ressemblants. On garde la correspondance entre le keypoint de l'image en cours et le meilleur keypoint de l'image de référence si l'inégalité suivante est respectée :

  $$d_1 < 0.7 \times d_2$$

  Avec d_1 (respectivement d_2) la distance au meilleur candidat (respectivement la distance au second meilleur candidat). Ce test permet d'éliminer les correspondances ambigues. On garde le point seulement si le premier choix est bien supérieur au deuxième.

- Si le nombre de correspondances valides est supérieur ou égal au minimum requis ($N_{\text{min}} = 4$), une matrice d'homographie est estimée. L'algorithme RANSAC (*Random Sample Consensus*) est utilisé avec un seuil de réajustement fixé à 4.0 pixels afin de rejeter les points aberrants résiduels.

- Comme RANSAC évalue les points d'un point de vue purement mathématique, il peut conserver des données physiquement irréalistes malgré les premiers filtrages. Une fonction de vérification géométrique supplémentaire est donc introduite pour les éliminer..

Celle-ci prend en compte la que la caméra d'étude est fixe, donc les photos ne doivent pas trop bouger. Ainsi des rotations, translations et changements d'échelle trop importants sont refusés. Les paramètres ont été fixés après tests.

- Si la matrice H valide l'ensemble de ces critères géométriques, une transformation de perspective (*Warp Perspective*) est appliquée sur l'image couleur d'origine pour l'aligner parfaitement sur la géométrie de la référence. Si l'homographie échoue, s'il y a un manque de points, ou si les critères géométriques ne sont pas respectés, l'image d'origine est renvoyée telle quelle dans le dossier de sortie et une alerte apparaît dans la console du terminal.

3 Traitement des images panoramiques

La plupart des images à étudier sont en format panoramique. Or, les différents plugins QGIS ou scripts Python existants ne permettent pas de les traiter correctement puisqu'ils supposent que toutes les perspectives sont rectilinéaires (aucune déformation, ni d'assemblage).

Il faut d'abord trouver la projection géométrique appliquée à notre panorama avant de le traiter [5]. La projection géométrique est la façon dont va être déformée l'image. Dans notre cas, on repère des lignes d'horizon bien droites, une largeur d'image très largement supérieure à sa hauteur, et on ne retrouve pas de distorsion aux bords. Il s'agit sûrement d'une projection cylindrique (la plus commune, elle enroule virtuellement le paysage autour d'un cylindre), assemblée par plusieurs photos. Même si ce type de projection donne un panorama semblant garder les proportions, les lignes et courbes sont déformées et donc les panoramas ne peuvent pas être traités correctement par les algorithmes existants [21]. La propriété mathématique que suppose les différents scripts de géoréférencement est que toute ligne droite dans le monde réel en 3D doit apparaître comme une ligne parfaitement droite sur la photo en 2D. Or, comme l'image est comme enroulée autour d'un cylindre, les lignes droites en 3D sur les bords sont courbées sur le panorama en 2D.

L'idée est alors de découper notre panorama sur la partie qui nous intéresse en réalignant les lignes (i.e. projection rectiligne) et donc avec un point de vue horizontal (FOV) standard de 80 degrés par exemple. Après quelques recherches bibliographiques, le logiciel FFmpeg semble correspondre à ce que l'on veut [4].

Voici un exemple type pour utiliser FFmpeg dans notre cas, donc avec une entrée en projection cylindrique et une sortie rectiligne :

```
ffmpeg -i 2023-06-14-12-00-03-001.jpg -vf "v360=input=cylindrical:
output=rectilinear:ih_fov=210:yaw=0:h_fov=80:v_fov=40" 06_14_centre.jpg
```

Avec :

`2023-06-14-12-00-03-001.jpg` L'image d'entrée à traiter.

`input=cylindrical` Définit la projection d'entrée comme cylindrique.

`output=rectilinear` Définit la projection de sortie comme rectiligne.

`ih_fov=210` Le champ de vision de l'image d'entrée de 210 degrés.

`yaw=0` L'angle d'inclinaison horizontal du centre de l'image de sortie par rapport au centre du panorama d'entrée. Ici à 0 degré pour rester au centre.

`h_fov=80` Le champ de vision horizontal de l'image de sortie à 80 degrés.

`v_fov=40` Le champ de vision vertical de l'image de sortie à 40 degrés.

`06_14_centre.jpg` Le nom du fichier de l'image de sortie.

4 Géoréférencement des photos

4.1 Photogeoref par Meteoexploration

Photogeoref est un script Python créé par Meteoexploration [11]. L'algorithme de géoréférencement utilisé dans Photogeoref "recrée" la géométrie de la formation de l'image de la caméra directement sur le MNT afin de générer une photo virtuelle du relief, vue depuis la position exacte de la caméra [2].

La première étape consiste à redéfinir les coordonnées du MNT. L'algorithme translate le repère, plaçant ainsi l'origine du système de coordonnées globales sur la webcam. Et tous les points du MNT sont recalculés par rapport à l'observateur [15].

Ensuite, l'algorithme pivote les axes du MNT (Nord, Est, Haut) pour qu'ils s'alignent avec l'orientation de la caméra. Pour faire ça, il calcule le vecteur de direction de vue en utilisant les coordonnées de la cible visée (milieu de la photo) et les coordonnées de la caméra. Il calcule ensuite les autres vecteurs directionnels avec des produits vectoriels. Finalement, une matrice de transformation est appliquée sur tous les points.

Une fois que les points du MNT sont bien orientés par rapport à la vue de la caméra, l'algorithme les projette sur une fenêtre de visualisation 2D correspondant à la surface du capteur de l'appareil, en utilisant les différents paramètres de celui-ci (distance focale, largeur, hauteur).

L'algorithme s'assure de ne pas afficher des pixels qui seraient situés derrière l'appareil photo. Il va "diviser" le MNT en deux par rapport à la caméra pour limiter le champ de vision à 180°. Pour ça, il va calculer le produit scalaire entre le vecteur pointant vers le centre de la photo et chaque vecteur reliant l'appareil à une cellule du MNT. Lorsque le produit scalaire est négatif, on est derrière donc on n'affiche rien.

Finalement, l'algorithme applique un petit ajustement vertical sur toutes les cellules du MNT pour compenser la courbure de la Terre.

4.2 Prise en main du logiciel et validation

Lorsqu'on lance le script Photogeoref une page s'ouvre avec les différents paramètres (position de la caméra, MNT, position du point cible, focal...) écrits dans un fichier *yaml*. Ensuite, le script applique l'algorithme décrit dans la partie précédente et c'est à ce moment que l'on peut régler les paramètres pour que la photo virtuelle du relief colle parfaitement avec la photo que l'on veut géoréférencer. Lorsqu'on est satisfait du résultat, on sauvegarde le photo projetée dans un fichier *tif*. Pour plus de détails sur le fonctionnement du script, ainsi que les modules Python nécessaires, voir le manuel utilisateur [11].



Figure 2: Comparaison entre la photo géoréférencée (à gauche) et l'image Planet (à droite) au 06/28/2023.

D'après la Figure 2, on remarque que le géoréférencement se superpose parfaitement à l'image Planet, surtout sur le versant qui nous intéresse (celui le plus proche de la caméra).

5 Segmentation de la neige

L'objectif de cette étape est de créer un masque neige/pas neige à partir d'une photo oblique. Plusieurs méthodes sont expliquées, les plus classiques par seuillage aux plus récentes en utilisant des modèles d'apprentissage profond (Deep Learning).

Ce type de segmentation est appelé segmentation sémantique. La segmentation sémantique crée un masque mettant en valeur les classes qui nous intéressent, pour notre cas on a qu'une classe : "neige". Elle ne différencie pas les individus au sein d'une même classe, mais attribue la même étiquette à toute la classe. Pour notre projet, la segmentation devra nous renvoyer, à partir d'une photo, une image en noir et blanc où le blanc représente la neige, le noir tout le reste.

5.1 Méthode classique par seuillage

De nombreux articles utilisent une méthode par seuillage pour classifier la neige sur les photos obliques [2, 3, 6, 16]. Le problème majeur est de réussir à trouver un seuil acceptable et généralisable malgré les variations lumineuses (zones ensoleillées vs zones d'ombre) afin d'isoler précisément la neige.

5.1.1 Explication de l'algorithme

Un simple seuillage global ne fonctionne pas car dépend trop de la situation de la photo (heure, nuages, ombre). Pour y remédier, l'image est d'abord convertie de l'espace de couleur standard BGR (Blue, Green, Red) vers l'espace CIE LAB (L pour la luminosité, A et B pour des gammes de couleurs). Un algorithme d'égalisation d'histogramme adaptative limitée en contraste (CLAHE — Contrast Limited Adaptive Histogram Equalization) [19] est ensuite appliqué uniquement sur le canal de luminance L pour ne pas modifier les couleurs. Cette méthode permet d'augmenter localement le contraste dans les zones sombres sans saturer les zones claires, et ainsi essayer d'égaliser les contrastes.

Une fois l'image harmonisée, elle est convertie dans l'espace HSV (H pour la teinte, S pour la saturation et V pour la valeur). Contrairement à l'espace RVB où la luminosité est partagée entre

les trois canaux, l'espace HSV permet d'isoler la pureté de la couleur (saturation) et sa brillance (valeur). La segmentation est réalisée en combinant deux masques distincts par une opération logique OU :

- Le masque "neige au soleil" : Il cible les pixels ayant une très forte brillance (Valeur élevée) et une très faible coloration (Saturation proche de zéro), ce qui correspond à la blancheur éclatante de la neige.
- Le masque "neige à l'ombre" : Il garde les zones de neige ombragées qui tendent généralement vers des teintes bleutées ou violacées.

Les valeurs retenues pour les trois paramètres (H, S, V) des deux masques ont été ajustées empiriquement.

Les nuages constituant une source d'erreur constante, l'application d'un masque de ciel s'avère souvent nécessaire. Cependant, la création manuelle d'un tel masque limite l'automatisation du processus : si cette approche est viable pour une caméra fixe, elle exige de définir un nouveau masque à chaque changement de capteur ou de point de vue.

5.1.2 Analyse des résultats

Malgré les hyperparamètres choisis à la main pour coller au plus avec les données, les résultats de la classification de la neige par seuillage ne sont pas satisfaisants. On peut voir Figure 3 deux exemples de l'algorithme, avec un masque enlevant le ciel comme expliqué à la fin du paragraphe précédent.

La photo Figure 3a montre un cas plutôt facile et courant, photo au Soleil et assez dégagé. Les patchs de neige sont bien détectés et les contours sont parfaits, mais les rochers au premier plan de l'image sont classifiés comme neige, Figure 3b.

La photo Figure 3c est un cas plus rare et totalement différent de l'autre photo. L'algorithme de classification ne fonctionne pas du tout et ne détecte pas la neige. En plus de ça, il détecte encore les rochers au premier plan, Figure 3d.

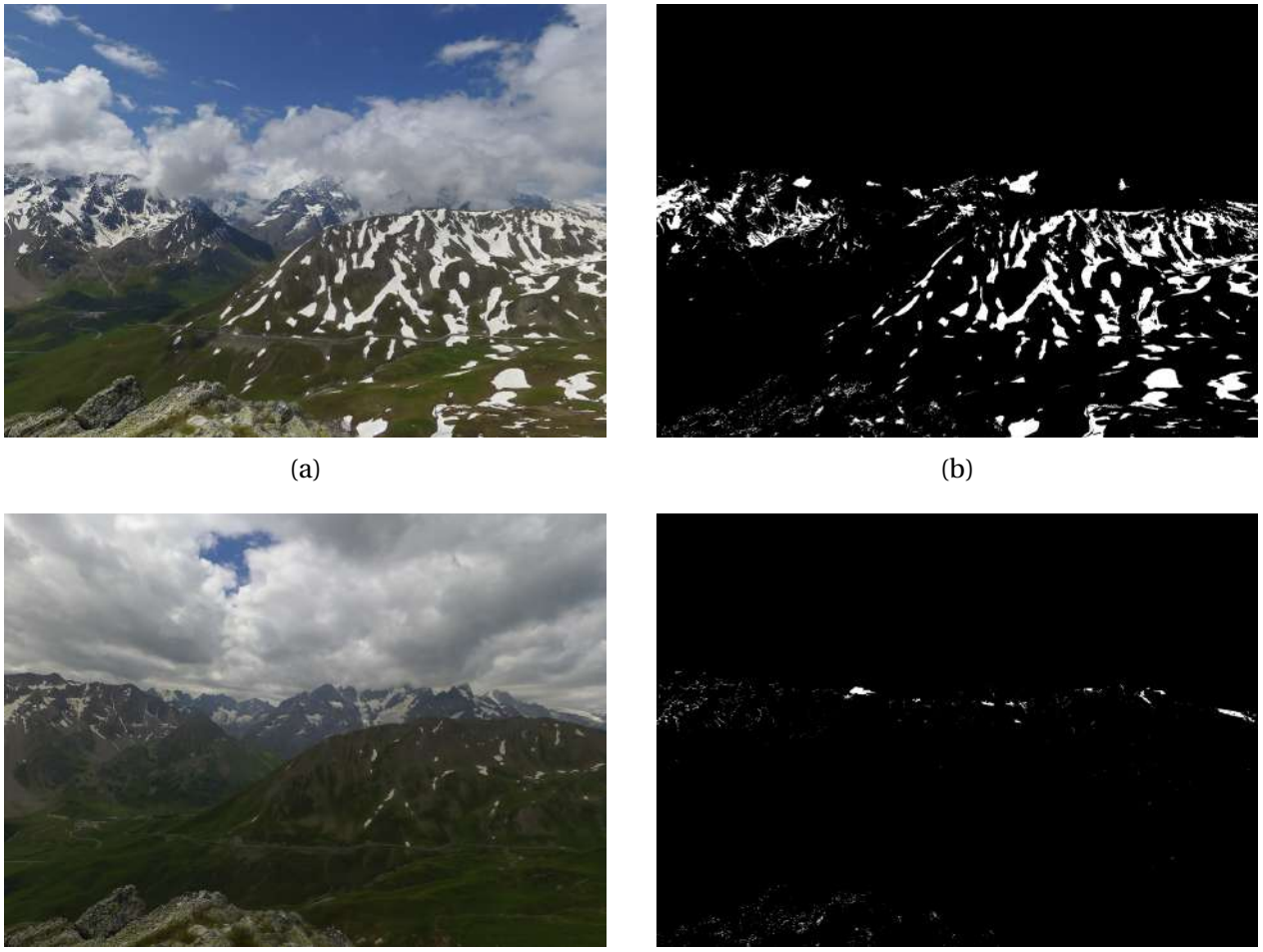
Un algorithme classique par seuillage ne peut pas bien fonctionner pour ce problème, les variations météorologiques l'empêchent. De plus, un masque annulant le ciel et les nuages a été passé ici, sans celui-ci les résultats sont encore moins bons car les nuages sont la plupart du temps classifiés comme neige. Ainsi, un algorithme de ce type n'est pas adapté à notre problème, malgré le bon nombre d'articles qui utilise ce type de méthode [2, 3, 6, 16].

5.2 Deep Learning pour la segmentation sémantique

Plusieurs modèles de Deep Learning ont été testés, mais toute la préparation des données reste identique, ainsi que le processus d'entraînement et de validation. La préparation des données, ou bien le processus d'entraînement et de validation, sont beaucoup inspirés de l'article [9].

5.2.1 Annotation et préparation des données

Les données ayant servi pour entraîner et évaluer les différents modèles proviennent en très grande partie du *MountainScape Segmentation Dataset* [17]. Les images, ainsi que leurs masques



(a) (b)

(c) (d)

Figure 3: (a) Photo découpée du 14/06/2023, (b) son masque correspondant avec l'algorithme par seuillage. (c) et (d) pareil mais au 07/07/2023.

de segmentation, étant disponibles, il a suffi de filtrer les classes non pertinentes (roche, arbre, herbe, etc.) pour se ramener à un problème de classification binaire : présence ou absence de neige. Au total, 70 paires d'images et de masques ont été sélectionnées, en essayant d'avoir chaque cas de figure (pas de neige, patchs de neige ou recouvrement partiel et total).

Ensuite, les images et leurs masques ont été découpés en utilisant la méthode de la fenêtre glissante (taille de 512×512 pixels, avec un pas de glissement, ou *stride*, de 256 pixels assurant un chevauchement entre les zones adjacentes) [9]. Cette opération a permis de générer un total de 6459 sous-images (ou patchs), réparties aléatoirement entre le jeu de données d'entraînement (environ 80 %, 4980) et le jeu d'évaluation (20 %, 1479).

Afin de limiter le fort déséquilibre de classes (les zones sans neige étant largement majoritaires), un sous-échantillonnage a été appliqué : seuls 8 % des patchs ne contenant aucune trace de neige ont été conservés, de manière aléatoire.

Enfin, pour améliorer la capacité de généralisation du modèle et limiter le surapprentissage, des opérations d'augmentation de données (retournements horizontaux et verticaux, rotations) ont été appliquées. Celles-ci ont été générées à chaque itération, et appliquées seulement sur le

jeu d'entraînement.

Avant d'être fournis au réseau, les patchs (dont les intensités de pixels sont initialement comprises entre 0 et 255) ont subi une étape de normalisation. Cette transformation est essentielle, elle centre les valeurs autour de 0 pour ne pas avoir des valeurs trop grandes, ce que les modèles n'aiment pas trop. Les paramètres de normalisation appliqués correspondent aux moyennes (0.485, 0.456, 0.406) et écarts-types (0.229, 0.224, 0.225) des canaux RGB du jeu de données ImageNet. L'architecture HRNet ayant été initialisée avec des poids pré-entraînés sur ImageNet, il faut appliquer ces poids. Cela permet au modèle d'exploiter directement tout ce qu'il a appris, que ce soit les couleurs ou bien les formes, les textures. Enfin, les masques, initialement en niveaux de gris, ont été binarisés (seuil à 127) pour s'assurer que chaque pixel appartienne à la classe positive (neige) ou négative (pas neige).

5.2.2 Environnement de travail

Toutes les expériences ont été réalisées sous macOS 26.5, en utilisant la bibliothèque PyTorch 2.12.1 avec le backend MPS (*Metal Performance Shaders*) pour l'accélération matérielle sur le GPU d'une puce Apple M4 Pro. Pour maximiser la vitesse d'entraînement et de validation, le pipeline a été optimisé afin de supprimer les goulots d'étranglement CPU-GPU. Les paramètres finaux retiennent une taille de lot (*batch size*) de 8 images, un chargement direct des données depuis le SSD, et l'allocation de 4 processus de travail ($num_workers = 4$) pour paralléliser la préparation des images.

Ce qui permet de réaliser, pour la structure HRNet, une itération en environ 1.2 secondes, soit environ 12 minutes par époque (une époque correspond à l'entraînement et évaluation sur le jeu de données entier). Ainsi un entraînement du modèle pendant 20 époques représente 4 heures, ce qui est tout à fait raisonnable.

5.2.3 Processus d'entraînement et d'évaluation

Les images d'entrée présentent un arrière-plan complexe avec beaucoup de détails et d'éléments différents, avec des patchs de neige à contours précis et des grandes étendues enneigées. Un mécanisme de supervision profonde (*Deep Supervision*) a été choisi [9]. Cette approche est essentielle pour garantir une délimitation précise des contours des patchs de neige, en préservant la détection des grandes zones enneigées. Tous les hyperparamètres sont identiques à ceux cités dans [9].

Le réseau HRNet génère deux sorties, une perte auxiliaire est introduite, extraite de la première étape de l'encodeur. Celle-ci va précisément servir à ce que le modèle fasse attention aux détails fins. La sortie principale est la prédiction finale du réseau [9].

Pour chacune des sorties, la fonction perte est la moyenne entre la composante BCE (Binary Cross-Entropy) qui analyse pixel par pixel la prédiction, et la composante Dice Loss qui analyse la forme globale des patchs de neige et très souvent utilisée en segmentation sémantique.

La fonction perte finale L est seulement une moyenne pondérée entre la fonction perte de chaque sortie. Elle s'écrit :

$$L = 0.7L_{main} + 0.3L_{aux}$$

L'optimisation des poids du modèle est réalisée par l'algorithme AdamW, sur une durée de 50 époques.

Un autre détail crucial de l'entraînement du modèle est le planificateur de taux d'apprentissage (Learning Rate Scheduler). Le learning rate règle la vitesse à laquelle le réseau apprend. S'il est trop lent, l'entraînement convergera très lentement (il faudra beaucoup d'époques pour avoir un résultat convenable), mais s'il est trop rapide, il va passer trop vite sur des détails et va stagner ou exploser. Le learning rate scheduler permet d'ajuster dynamiquement le taux d'apprentissage, on fixe un learning rate au début (ici 10^{-4} comme dans [9]) et lorsque le modèle stagne sur plusieurs époques, le scheduler va baisser le learning rate pour faire progresser le modèle.

Pour plus de détails sur cette partie (métrique IoU, réseau HRNet, optimiseur AdamW), voir Annexe B.

5.2.4 Prédiction

Pour la prédiction, les étapes sont moins nombreuses que pour l'entraînement. D'abord on charge le modèle déjà entraîné et on crée notre matrice de normalisation étant la même que précédemment. L'image à prédire subit les mêmes traîtements que celles utilisées dans le jeu de données d'évaluation, ie qu'elle est découpée en carrées de 512×512, puis la prédiction est appliqué sur chacun d'entre eux séparément.

Une fois que les images sont prêtes et dans la même configuration que celles de l'entraînement, la prédiction peut commencer. Comme quasiment chaque carré découpé (sauf ceux aux bords de l'image) chevauche un autre (en fait au centre de l'image il y a un chevauchement de 4 carrés), alors la plupart des pixels sont prédits plusieurs fois. Pour gérer ça, la prédiction finale utilise une technique de Soft Voting. Cette technique enregistre les probabilités qu'un pixel soit enneigé ou non dans les différents carrés, puis fait une moyenne des probabilités (il y en a 4 au plus). Ensuite si celle ci est supérieure à un seuil, ici $0,4$, alors le pixel renvoyé sera blanc sur le masque, ie enneigé. Cette méthode de Soft Voting permet de limiter les erreurs aux bords, puisqu'un pixel se trouvant sur un bord d'un carré se situera au centre du carré adjacent.

5.2.5 Améliorations possibles

Le modèle final n'est pas parfait ($mIoU \simeq 0.7$), mais quelques pistes d'amélioration sont possibles. Le principal problème est le jeu de données qui n'est pas assez diversifié, certaines situations ne sont que très peu présentes (photos recouvertes de neige par exemple) et donc la segmentation dans ces cas là est très mauvaise car le modèle n'a pas vu assez d'échantillons en entraînement.

De plus, un modèle de transformer comme SegFormer pourrait donner des résultats bien meilleurs [7], mais le portage sur MacOs n'est pas fait, et donc l'entraînement du modèle prendrait trop de temps puisque le backend MPS n'est pas pris en charge.

6 Rendu final

L'objectif de ce stage est de construire un mini logiciel ou script que les différents chercheurs/ingénieurs pourront utiliser. Ainsi, il a fallu tout automatiser et simplifier pour que celui-ci soit facile à prendre en main. Le projet ainsi qu'un exemple et le manuel d'utilisation sont présents sur Git, au lien suivant : https://github.com/Placido18/GeoSnow.

6.1 Fonctionnement du script

Le mini logiciel et ses fonctionnalités sont décrites en détail dans le manuel utilisateur présent dans le lien Git, mais l'architecture est décrite ici. Il prend en entrée un dossier d'images brutes issues de la même caméra, et renvoie en sortie les masques de neige géoréférencés prêts à être intégrés dans un logiciel SIG (Système d'Information Géographique) comme QGIS.

Le programme a été conçu autour d'un script principal (`GeoSnow.py`) utilisant la bibliothèque `argparse` permettant à l'utilisateur de lancer toujours le même programme en pouvant changer les paramètres. Il permet ainsi d'enchaîner toutes les étapes du logiciel.

Le flux de traitement complet, qui peut être entièrement automatisé via l'argument `-a`, se décompose comme suit :

- **Reprojection et découpage** grâce à FFmpeg, et aux paramètres de découpage à rentrer dans le fichier des paramètres. Il faut rajouter l'option `-d`.
- **Ouverture de l'interface de géoréférencement** permettant de superposer la `masterimage` par rapport au MNT projeté. L'interface est celle de Photogeoref.
- **Recalage géométrique** des photos du dossier d'entrée par rapport à une image de référence (la `masterimage` définie dans la configuration) pour corriger les micro-mouvements. Cette étape est ignorée si l'utilisateur traite une image unique en utilisant l'argument `-u`.
- **Segmentation de la neige** : Le programme génère des masques binaires isolant la neige en appliquant le modèle d'apprentissage profond défini précédemment.
- **Géoréférencement** des masques grâce à l'outil Photogeoref, le script projette les masques sur un Modèle Numérique de Terrain (MNT) et génère des fichiers finaux au format `.tif`.

6.2 Paramétrage et exécutions isolées

Pour s'adapter à différentes situations, plusieurs options peuvent être ajoutées et tout peut être configuré :

Interfaces de calibration visuelle : L'outil propose des interfaces dédiées (`-ig` et `-id`) permettant d'ajuster visuellement les paramètres de géoréférencement et de découpage. A noter que lorsqu'on lance le programme avec l'option -a, l'interface de géoréférecement se lance automatiquement.

Fichier de configuration : L'ensemble des données nécessaires aux algorithmes (coordonnées de la caméra, inclinaison, champ de vision ...) est écrit dans un fichier YAML unique, situé par défaut dans `./settings/georefparam.yml`.

Gestion des fichiers intermédiaires : L'utilisateur peut choisir de sauvegarder les rendus de chaque étape de traitement (découpage, masque non projeté) dans des dossiers dédiés grâce à l'argument `-i`.

Exécution des modules individuellement : Chaque module du processus (découpage, recalage, segmentation, géoréférencement) peut être appelé de manière autonome, et les dossiers d'entrée et de sortie peuvent être personnalisés.

Dossiers standards : Si l'utilisateur n'indique pas de chemins de fichiers personnalisés lors de l'appel individuel d'un module, le programme va récuperer les images dans le dossier `./input_images/` et distribuer les résultats dans des sous-dossiers spécifiques comme `./output/decoup/` ou `./output/segmentation/`.

7 Analyse des résultats

Cette partie est consacrée à valider et/ou relever les erreurs du logiciel sur d'autres photos que celles utilisées pour créer et tester tous les algorithmes. Ces photos sont tirées de la même caméra que précédemment mais sur une année entière pour voir le résultat sur différentes saisons et donc sur différents taux d'enneigement et paysages. 25 photos ont été sélectionnées, de juillet 2025 à août 2026 en ayant environ 2 photos par mois. Ce qui permet d'avoir toutes les saisons et donc de voir si certains cas entraînent plus d'erreurs que d'autres. L'analyse a été effectuée sur le recalage et la création des masques binaires, car ce sont les deux seules étapes qui peuvent introduire des erreurs non corrigeables par l'utilisateur. L'étape du géoréférencement est la plus délicate pour l'utilisateur si les paramètres sont bien réglés alors les photos seront bien projetées.

7.1 Évaluation au fil des saisons

L'observation des masques générés sur l'année met en évidence plusieurs comportements du modèle :

En période estivale (neige quasiment inexistante) : C'est la situation la plus représentée parmi les photos utilisées pour entraîner le modèle de segmentation, ainsi les masques générés sont très précis. Quelques erreurs subsistent, les roches très claires (notamment celle au premier plan sur la photo) engendrent encore des faux positifs, tout comme certaines délimitations montagne/ciel , Figure 4. Cependant ces erreurs sont très peu présentes et la détection reste très fiable.

Au printemps et à l'automne (neige fragmentée) : Cette situation peut sembler la plus complexe à cause de la délimitation de chaque patch de neige. Or, on peut voir que le modèle gère parfaitement les patchs de neige et les contours sont précis, Figure 5. Cette efficacité s'explique par les nombreuses photos de ce type lors de l'apprentissage. La comparaison Figure 5 nous montre aussi que les nuages ne sont jamais détectés comme neige. Cependant, on remarque que lorsqu'il y a une étendue de neige trop importante, le modèle ne la détecte pas correctement. On va voir ce phénomène encore plus dans la prochaine comparaison, lorsque l'image entière est enneigé.

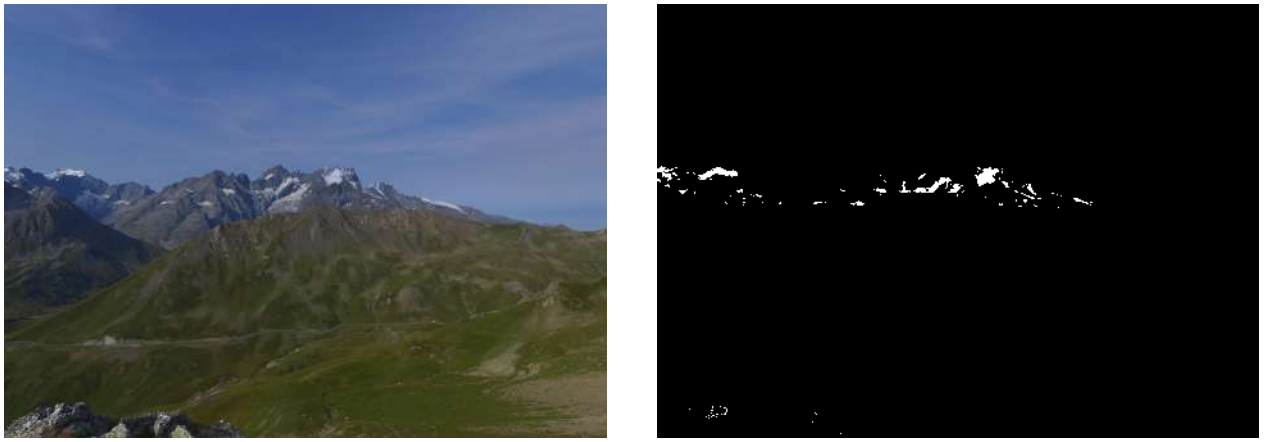


Figure 4: Photo découpée du 06 août 2025 et son masque binaire correspondant.



Figure 5: Photo découpée du 01 juin 2026 et son masque binaire correspondant.

En hiver (recouvrement total) C'est dans cette configuration que le modèle échoue, générant des masques quasi intégralement noirs malgré un enneigement total (Figure 6). Ce problème de faux négatifs découle directement du sous-échantillonnage de paysages hivernaux dans les données d'entraînement. Le réseau confond fort probablement ces vastes étendues blanches et uniformes avec une couverture nuageuse.

En général, les masques générés durant des périodes où l'enneigement n'est pas total sont fiables. Les nuages ne sont pas confondus avec la neige et les patchs de neige sont délimités avec précision. Cependant, il faut améliorer le modèle pour des cas où l'enneigement est totale car les masques générés sont absurdes.

7.2 Analyse de l'algorithme de recalage

Tout comme pour l'étape de segmentation, le recalage (SIFT) échoue pour des photos où la neige recouvre tout le paysage. En effet, l'algorithme ne trouve pas assez de points (moins de 4) de correspondance entre l'image de référence et l'image courante à recaler par manque de



Figure 6: Photo découpée du 02 février 2026 et son masque binaire correspondant.

contraster et de textures rocheuses. Ainsi, la photo telle quelle est renvoyée et donc les légères perturbations (rotation, translation) restent.

8 Bilan personnel

Ce stage a été enrichissant, sur le plan technique et aussi personnel. Il m'a permis de découvrir le monde de la recherche et de mener un projet important quasiment exclusivement seul.

Défis techniques et méthodologiques : Le principal défi de ce stage a été l'étape de segmentation. N'ayant jamais conçu de modèle de *Deep Learning* auparavant, j'ai dû me former sur ce sujet. Le plus frustrant dans la création du modèle était de lancer des entraînements pendant une journée entière pour finalement constater que les performances stagnaient, voire se dégradaient. Voulant enrichir mon jeu d'entraînement pour pallier ce problème, j'ai tenté de créer des masques binaires à la main, mais obtenir des masques binaires parfaitement contourés est impossible humainement.

Sur le plan méthodologique, la gestion du temps a constitué un véritable apprentissage. Le cadre de travail en laboratoire de recherche est très peu directif, ce qui offre une grande liberté mais exige une forte autodiscipline et une certaine motivation. Par exemple, j'ai pu passer énormément de temps à essayer d'intégrer et de faire fonctionner des codes existants (comme le modèle PyLC), pour finalement me rendre compte que la piste n'était pas concluante. J'ai donc appris qu'il ne fallait pas passer trop de temps à rester bloquer sur une piste.

Moyens mis en œuvre et leçons tirées : Pour surmonter ces obstacles technologiques, j'ai du beaucoup me documenter. En m'appuyant beaucoup sur des articles scientifiques et des sites spécialisés, j'ai pu débloquer la majorité de mes problèmes d'implémentation. Avec le recul, je réalise que je n'ai pas assez sollicité plus régulièrement mes tuteurs de stage, par peur de déranger. Ce stage m'a fait comprendre qu'il ne faut pas hésiter à demander de l'aide à n'importe qui, car ça permet souvent de gagner beaucoup de temps.

Compétences acquises : Malgré ces difficultés, j'ai acquis de nombreuses compétences. Sur le plan technique, j'ai acquis de solides compétences en vision par ordinateur et en intelligence artificielle. Je suis désormais capable de manipuler le *framework* PyTorch, de concevoir un pipeline de traitement d'images complexe, et de comprendre les architectures de réseaux de neurones. J'ai également découvert un nouveau domaine scientifique et compris les enjeux biologiques liés au suivi des écosystèmes alpins. Sur le plan interpersonnel, cette expérience a considérablement forgé mon autonomie et ma persévérance. J'ai appris à mener un projet de bout en bout en m'adaptant aux imprévus, me préparant ainsi efficacement à mes futures expériences professionnelles.

9 Conclusion et perspectives

Ce stage avait pour ambition de fournir aux chercheurs une alternative automatisée à l'imagerie satellitaire pour le suivi journalier de la couverture neigeuse. Face aux limites des algorithmes classiques par seuillage, très sensibles aux variations d'éclairage et aux nuages, l'implémentation d'un modèle d'apprentissage profond (HRNet) pour créer les masques binaires a été testée.

Le logiciel développé, intégrant la correction panoramique, le recalage dynamique par points d'intérêts (SIFT), la segmentation sémantique et le géoréférencement 3D, montre qu'il est possible d'extraire des données précises à partir de simples webcams d'observation. Ces masques vont permettre d'étudier plus précisément la biodiversité à travers la dynamique locale de l'enneigement.

Au terme de ce projet, plusieurs perspectives d'amélioration se dégagent :

- **Sur la segmentation :** Le problème principal pour un écologue est de sous-estimer les étendues de neige au printemps, car le taux d'enneigement est très incertain (en comparaison avec l'hiver où la neige est présente partout). L'intégration de modèles basés sur des Transformers, tels que SegFormer, pourrait améliorer la fiabilité de la segmentation. Cependant, enrichir le jeu de données d'entraînement avec des cas extrêmes (brouillard intense, images partiellement masquées par la neige sur l'objectif) permettrait d'augmenter la précision du modèle de segmentation de manière significative. Ajouter des photos totalement enneigées en apprentissage est le plus urgent pour avoir un modèle fiable pour chaque saison.

- **Sur le recalage :** L'algorithme de recalage doit être amélioré pour qu'il fonctionne sur des photos où la couverture neigeuse est totale. Une idée serait de prendre deux photos de référence, une en été et une en hiver. Ensuite une partie des photos (celles où le recouvrement n'est pas total) sera recalée par rapport à la première image de référence et l'autre partie (les photos totalement enneigées) par rapport à la deuxième image. Le problème de cette méthode est que ça rajoute des paramètres à rentrer par l'utilisateur, ce qui rend le programme moins automatique.

- **Sur le géoréférencement :** Le géoréférencement est entièrement basé sur le script Photogeoref présenté précédemment. Or, la précision de celui-ci est uniquement basée sur celle de l'utilisateur lors du calibrage. Une précision optimale est alors impossible. Il faudrait explorer d'autres méthodes, par exemple basées sur des points de contrôle pour voir si c'est plus pertinent.

Ensuite, tester le logiciel sur d'autres caméras permettrait de relever d'autres problèmes, et donc des possibles pistes d'amélioration.

Annexes

A Algorithme SIFT

Formalisé par David Lowe en 2004 [10], SIFT (Scale-Invariant Feature Transform) est une méthode fondamentale en vision par ordinateur qui permet de détecter et de décrire des points d'intérêt (keypoints) robustes au sein d'une image. L'algorithme crée des identifiants uniques (descripteurs) pour chaque keypoint qui vont être invariants aux transformations géométriques et lumineuses. L'avantage de SIFT est qu'il est invariant par changement d'échelle, il arrive à détecter le même keypoint sur différentes images à différentes échelles.

La méthode SIFT est décrite en quatre étapes [10, 14, 21] :

- La première étape identifie des points d'intérêt potentiels. L'image d'origine est floutée à plusieurs degrés de flou par des filtres gaussiens. Ensuite, l'algorithme soustrait les images floutées adjacentes deux à deux pour faire ressortir des points d'intérêt (seuls les forts contrastes resteront après soustraction). Finalement, chaque pixel d'une image soustraite est comparé à ses voisins (du même plan mais aussi des plans supérieur et inférieur), et il est retenu comme potentiel point d'intérêt si c'est un extremum local strict.

- Cette étape élimine les keypoints où l'intensité n'est pas assez élevée. Aussi, les points faisant partis d'un "pont" (d'une ligne à fort contraste, comme la crête d'une montagne) sont éliminées à l'aide de l'étude spectrale d'une matrice Hessienne représentant la courbe.

- Une orientation est assignée à chaque keypoint pour que l'algorithme soit invariant par rotation.

- La dernière étape est la création du descripteur pour chaque keypoint. Un voisinage de 16 x 16 autour du point est pris. Ce voisinage est divisé en plusieurs blocs et pour chaque bloc, un histogramme d'orientation est calculé. Tous les histogrammes réunis forme le descripteur du point.

B Détails sur le processus d'entraînement du modèle de Deep Learning

B.1 Intersection over Union - IoU [8, 22]

L'IoU (Intersection over Union), aussi appelée indice de Jaccard, est une métrique permettant d'évaluer les performances des modèles de détection et de segmentation sémantique. Elle quantifie la précision des prédictions en mesurant le rapport entre la surface de chevauchement (l'intersection des pixels identifiés comme "neige" entre la prédiction et la vérité terrain) et l'aire totale couverte par la prédiction et la vérité terrain (l'union). La formule mathématique s'écrit :

$$IoU = \frac{neige_{pred} \cap neige_{reel}}{neige_{pred} \cup neige_{reel}} \in [0,1]$$

Où l'intersection représente la somme des intersections de chaque pixel dans chaque ensemble. De même pour l'union.

Plus l'IoU est proche de 1, plus le modèle est performant. En pratique une IoU égale à 1 est impossible, il y aura toujours quelques pixels qui seront mal classés. L'objectif est de la maximiser.

Cette métrique est choisie pour les problèmes de segmentation sémantique car elle est punit très bien les pixels mal placés, et se concentre sur le placement exact de ceux-ci.

Elle est invariante à l'échelle, ie l'IoU tiendra compte des dimensions totales de l'image. Un écart de 5 pixels fera baisser beaucoup plus l'IoU dans une image à 100 pixels que dans une image à 10 000.

En pratique pour la segmentation, une IoU entre 0.5 et 0.75 est bonne, le modèle identifie bien la neige mais il peut y avoir quelques problèmes aux frontières. Une IoU supérieure à 0.75 est excellente, le modèle prédit très bien la neige, avec des écarts mineurs de pixels.

B.2 Présentation de HRNet [23]

HRNet, raccourci de High-Resolution Net, est un réseau de neurones convolutif inventé en 2019 par des chercheurs de Microsoft.

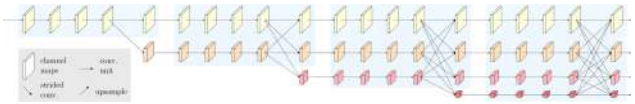


Figure 7: Corps principal du réseau HRNet. Source : Microsoft Research Blog [23]

La particularité de ce réseau est qu'il travaille avec plusieurs branches en parallèle, et donc conserve l'image à plusieurs degrés de résolution en même temps. Ce qui lui permet de voir les grandes zones (basse résolution), mais aussi les détails fins (haute résolution). Les flèches en diagonale permettent au modèle de faire communiquer les différentes échelles.

À ce corps principal, le modèle de segmentation sémantique (utile pour notre cas) ajoute une tête qui concatène les dernières 4 couches (celles toute à droite sur la Figure 7). En faisant comme ça, le modèle obtient une image finale qui contient les détails mais aussi la tendance globale du paysage. Ce modèle, adapté à la segmentation sémantique, s'appelle HRNetV2.

Références bibliographiques

[1] Isabelle Bloch. *Recalage d'images 2D et 3D*. https://perso.telecom-paristech.fr/bloch/VOIR/recalageVOIR.pdf. Cours en ligne (cit. on p. 6).

[2] Javier G. Corripio. "Snow surface albedo estimation using terrestrial photography". In: *The Cryosphere / Arolla, ETHZ* (2002). URL: https://www.arolla.ethz.ch/georef/albedo.pdf (cit. on pp. 4, 5, 8–10).

[3] Roman Fedorov et al. "Estimating Snow Cover From Publicly Available Images". In: *IEEE Transactions on Multimedia* 18.6 (2016), pp. 1187–1200. DOI: 10.1109/TMM.2016.2535356 (cit. on pp. 5, 9, 10).

[4] FFmpeg. *Documentation des filtres de FFmpeg*. https://ffmpeg.org/ffmpeg-filters.html. Documentation officielle (cit. on pp. 5, 7).

[5] Arnaud Frich. *Les projections géométriques*. https://www.guide-photo-panoramique.com/projections-geometriques.html. Guide Photo Panoramique (cit. on pp. 5, 7).

[6] S. Härer, M. Bernhardt, and K. Schulz. "PRACTISE – Photo Rectification And Classifica-TIon SoftwarE (V.2.1)". In: *Geoscientific Model Development* 9 (2016), pp. 307–321. DOI: 10.5194/gmd-9-307-2016. URL: https://gmd.copernicus.org/articles/9/307/2016/gmd-9-307-2016.pdf (cit. on pp. 5, 9, 10).

[7] Hugging Face. *SegFormer - Transformers Documentation*. https://huggingface.co/docs/transformers/model_doc/segformer. Consulté le 7 juillet 2026. 2021 (cit. on pp. 5, 13).

[8] Justin Johnson. *Deep Learning for Computer Vision - Lecture 13 about Object Detection*. University of Michigan. 2022. URL: https://web.eecs.umich.edu/~justincj/slides/eecs498/WI2022/598_WI2022_lecture13.pdf (cit. on p. 20).

[9] Yulin Liu et al. "IceSnow-Net: A Deep Semantic Segmentation Network for High-Precision Snow and Ice Mapping from UAV Imagery". In: *Remote Sensing* 17.17 (2025). ISSN: 2072-4292. DOI: 10.3390/rs17172964. URL: https://www.mdpi.com/2072-4292/17/17/2964 (cit. on pp. 5, 10–13).

[10] David G. Lowe. "Distinctive Image Features from Scale-Invariant Keypoints". In: *International Journal of Computer Vision (IJCV)* 60.2 (2004), pp. 91–110. URL: https://www.cs.ubc.ca/~lowe/papers/ijcv04.pdf (cit. on pp. 5, 20).

[11] *Manuel Photogeoref*. meteoexploration.com. URL: https://www.meteoexploration.com/python/photogeoref/photogeorefusermanual.pdf (cit. on pp. 5, 8).

[12] Gillian Milani. "Pic2Map: intégration de photographies dans QGIS". Rapport de Master. École Polytechnique Fédérale de Lausanne (EPFL), July 2014. URL: https://documents.epfl.ch/groups/l/la/lasig-unit/www/pic2map/rapport_PdM_gmilani.pdf (cit. on p. 5).

[13] Mountain Legacy Project. *PyLC (Python Landscape Classifier)*. Dépôt GitHub. Développement continu de l'outil original de Spencer Rose. 2024. URL: https://github.com/MLP-Hub/pylc_mlp (cit. on p. 5).

[14] OpenCV. *Introduction to SIFT*. https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_feature2d/py_sift_intro/py_sift_intro.html. Documentation officielle d'OpenCV 2.4 (cit. on pp. 5, 20).

[15] Jean-José Orteu. *Calibrage géométrique d'une caméra ou d'un capteur de vision stéréoscopique*. https://perso.univ-lemans.fr/~fvaret/opi/cours_maj/OPI_fr_M04_C01_web_gen_auroraW/co/Contenu01.html. Cours en ligne (cit. on p. 8).

[16] C. Portenier et al. "Towards a webcam-based snow cover monitoring network: methodology and evaluation". In: *The Cryosphere* 14.4 (2020), pp. 1409–1423. DOI: 10.5194/tc-14-1409-2020. URL: https://tc.copernicus.org/articles/14/1409/2020/ (cit. on pp. 4–6, 9, 10).

[17] Mountain Legacy Project. *MountainScape Segmentation Dataset*. Version V3. 2024. DOI: 10.5683/SP3/CEYU10. URL: https://doi.org/10.5683/SP3/CEYU10 (cit. on pp. 5, 10).

[18] Spencer Rose. "An evaluation of deep learning semantic segmentation for land cover classification of oblique ground-based photography". MA thesis. University of Victoria, 2020. URL: http://hdl.handle.net/1828/12156.

[19] Siril Team. *Égalisation d'histogramme adaptative limitée en contraste (CLAHE)*. Siril Documentation. 2026. URL: https://siril.readthedocs.io/fr/latest/processing/clahe.html (cit. on pp. 5, 9).

[20] StayTechRich. *Computer vision 001: color detection with OpenCV*. Medium. 2019. URL: https://medium.com/@staytechrich/computer-vision-001-color-detection-with-opencv-58426c880449.

[21] Richard Szeliski. *Computer Vision: Algorithms and Applications*. 2nd. Springer, 2022. URL: https://szeliski.org/Book/ (cit. on pp. 4, 6, 7, 20).

[22] TarmacView. *Intersection sur Union (IoU) - Airport Lights Inspection*. 2024. URL: https://www.tarmacview.com/fr/glossary/intersection-over-union/ (cit. on p. 20).

[23] Jingdong Wang, Ke Sun, and Bin Xiao. *High-Resolution Network: A universal neural architecture for visual recognition*. Microsoft Research. 2020. URL: https://www.microsoft.com/en-us/research/blog/high-resolution-network-a-universal-neural-architecture-for-visual-recognition/ (cit. on pp. 5, 21).